



Lilian Weng 
@lilianweng



A super long overdue (3+ years?) post on scaling laws.

Compute is expensive. Scaling laws are a way to help us reason about the optimal compute allocation between data and model size before committing to a large run.

The post covers what scaling laws predict, how compute-optimal allocation works, why Kaplan et al. and Chinchilla disagree, and how data limits + fitting details make extrapolation tricky. 一篇严重拖延（超过3年？）的关于缩放定律的文章。

计算成本高昂。缩放定律是一种方法，能帮助我们在投入大规模运行之前，合理推算出数据和模型规模之间的最优计算资源分配。

文章涵盖缩放定律的预测内容、计算最优分配的原理、Kaplan等与Chinchilla的分歧原因，以及数据限制与拟合细节如何让外推变得困难。



lilianweng.github.io

Scaling Laws, Carefully 缩放定律，谨慎解读

Scaling laws are one of the most critical empirical findings in deep learning. The observation is simple in form: the training loss $\mathcal{L}$ decreases predictably as we scale up model size N , dataset... 缩放定律是深度学习中最重要实证发现之一。其表述形式简洁明了：随着模型规模 N 和数据集的扩大，训练损失 $\mathcal{L}$ 会呈现可预测的下降趋势...

4:06 AM · Jun 26, 2026 · **84.8K** Views

博客链接: <https://lilianweng.github.io/posts/2026-06-24-scaling-laws/>

翁荔表示，这篇关于 [Scaling Law](#) 的博客文章迟到了 3 年多，而这篇文章涵盖了 Scaling Law 的预测内容、计算最优分配的原理、Kaplan 等与 Chinchilla 的分歧原因，以及数据限制与拟合细节如何让外推变得困难等内容。

对此，网友们排队表示欢迎：

- 

Maya Ravichandran 玛雅·拉维昌德兰  @maya_rav · 1h
Yay!! We love Lil'log! 耶!! 我们爱小日志!

 23
- 

Yusuf 优素福  @YusufAfifi3 · 24m
Good read 不错的读物

 1
- 

Wajahat @wajahatalikhan_ · 35m
love your technical blogs... 很喜欢看你的技术博客...

 6
- 

rez @residual00 · 2h
Goat is back 大神回归

 63
- 

lily zhang  @lily_gpupoor · 2h
Lil'Log is so back ! Lil'Log 强势回归!

 1 81
- 

Simon T 西蒙·T @l_donciccanplay · 1h
LET'S GO 咱们走吧

 14
- 

Hasan Faisal @HasanFaisall · 4h
thanks for making my day! 让我今天心情大好!

 1 367

在分享博客的推文下方，翁荔自己评论感叹说，想必很多人会直接让 AI 来总结这篇文章，而不是真正地阅读这篇文章。



她还进一步表示，她很快还会设置一个模型来自动更新 Lil'Log。

机器之心也在 AI 的辅助下对翁荔大神这篇新作进行了整理翻译：

谨慎对待 Scaling Law

Scaling law 是深度学习领域中最关键的经验发现之一。这一观察结果的形式很简单。随着我们扩大模型规模 N 、数据集规模 D 和计算量 C ，训练损失会以可预测的方式降低。这种降低趋势遵循幂律曲线，在双对数坐标图上表现为一条直线。我们可以将 **Scaling law** 视为一个框架。它描述了计算量、损失、模型规模与数据之间的关系。其核心在于如何将宝贵的计算资源在 N 和 D 之间进行最优分配。

这种可预测性让 **Scaling law** 在实践中具有极高的价值。一种常见的工作流程是，在少量小型训练任务上拟合 **Scaling law**，然后进行外推，以此估算更大模型所需的 **token** 数量和计算资源。

Symbol	Note
N	Model size, measured in parameter count.
D	Training dataset size, usually measured in token count.
C	Training compute in FLOPs. As a useful approximation, $C \approx 6ND$ (Kaplan et al. 2020), where $2ND$ accounts for the forward pass and $4ND$ for backpropagation.
E	Irreducible loss
$L, \hat{L}(\cdot)$	Test loss / test loss prediction function; can also refer to training loss, since they are strongly correlated.
ϵ	Generalization error.

早期阶段：机器学习损失的可预测性

在 **Scaling law** 成为主流概念之前，研究人员就已经探讨过泛化误差随规模变化的可预测性。

Amari et al. (1992) 使用贝叶斯方法⁹和退火近似推导出了四种类型的学习曲线。

1. **确定性学习算法，无噪声数据，具有唯一解：** $\epsilon \sim c \cdot D^{-1}$ ，其中 c 是某个常数。
2. **确定性学习算法，无噪声数据，具有多个等效解：** $\epsilon \sim c \cdot D^{-2}$ 。随着新数据点的加入，学习速度会加快。由于模型只需要学习参数的最优流形，无需寻找单一的解空间点。
3. **确定性学习算法，含噪数据：** $\epsilon \sim c \cdot D^{-1/2}$ 。数据中的噪声会增加学习难度。
4. **随机学习算法，含噪数据：** $\epsilon \sim c \cdot D^{-1} + E$ 。这里的不可约损失 E 是指随机学习器无法进一步降低的残差错误，例如当模型在大型数据上容量耗尽时。

这四种类型的学习曲线都遵循幂律：

$$\epsilon \sim c \cdot D^\alpha + E$$

其中 E 可以为 0，并且 $\alpha = -2, -1, -1/2$ 。虽然他们的理论设定基于简化的二分类任务，但这也为构建经验性的机器学习损失预测模型指明了有用的方向。

早期的一项实证研究（Hestness et al. (2017)）解释了泛化误差、模型规模和数据之间的关系。对于给定的训练数据规模，他们通过网格搜索确定最佳拟合的模型规模，然后绘制损失与训练数据集规模的关系图。在深度学习的四个不同领域（神经机器翻译、图像分类、语言建模和语音识别）中，研究人员观察到了一个反复出现的模式：

- 泛化误差作为一组因素（如数据规模）的幂律函数进行缩放。
- 模型的改进会改变误差曲线的位置，但似乎并未改变幂律指数。
- 有趣的是，架构的变化会改变幂律拟合的截距 (E)，但对指数 (α) 基本没有影响。幂律的斜率似乎是问题领域的属性，与模型架构关系不大。
- 拟合规模为 D 的数据集所需的大致模型参数 N 的数量同样遵循幂律缩放。

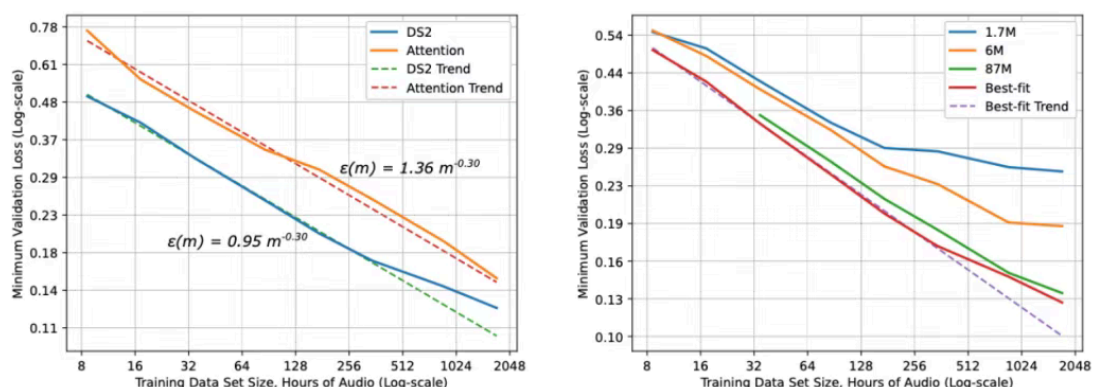


图1: Deep-Speech-2 (DS2) 与注意力语音模型的学习曲线（左图），以及各种规模的 DS2 模型的学习曲线（右图）。

当训练数据量变得很大时，小模型的损失会趋于平缓。（图片来源：Hestness et al. 2017）

幂律学习曲线各阶段的概念图解分为三个阶段。在**小数据区域**，当没有足够的学习信号时，模型的表现仅略好于随机猜测。在**中间的「幂律区域」**，我们观察到了损失、数据与模型规模之间的幂律关系。最后的**不可约误差区域**可归因于数据中的噪声等因素。

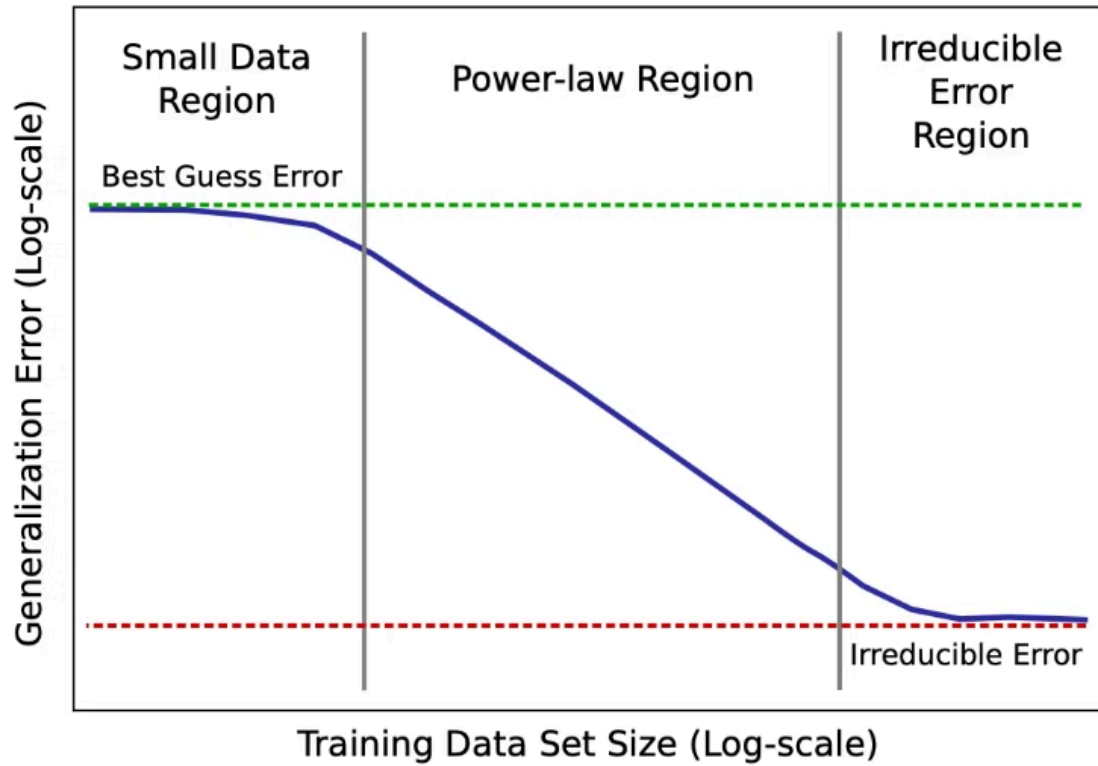


图2: 幂律学习曲线各阶段的概念图解。(图片来源: Hestness et al. 2017)

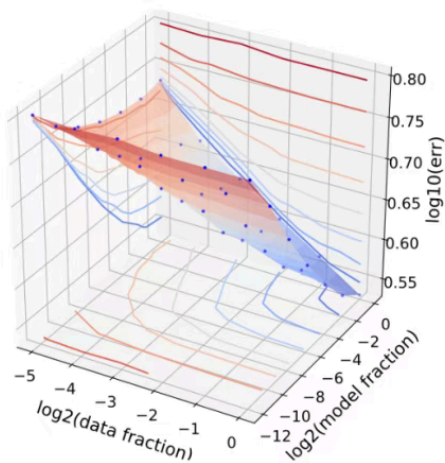
Rosenfeld et al. (2020) 进一步推进了这项工作。他们试图将误差建模为模型规模 N 和数据规模 D 的联合函数。这项研究跨越了多种不同的架构 (ResNet、WRN、LSTM、Transformer) 和优化器 (Adam、SGD 变体)。在经验上, 他们观察到, **保持一个轴不变时, 误差在另一个轴上按幂律衰减**:

$$\hat{L}(D, N) \approx \frac{A}{N^\alpha} + E_N, \quad \hat{L}(D, N) \approx \frac{B}{D^\beta} + E_D$$

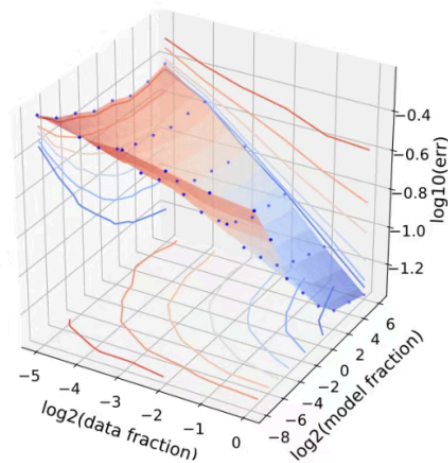
这些可以组合成一个联合形式:

$$\hat{L}(D, N) \approx \frac{A}{N^\alpha} + \frac{B}{D^\beta} + E$$

其中 $A > 0, B > 0, \alpha \geq 0, \beta \geq 0$ 是标量常数, 且 E 既不依赖于 N 也不依赖于 D 。



(a) Wiki103 error (cross entropy) landscape.



(b) CIFAR10 error (top1) landscape.

图3: 在双对数坐标系下, 数据规模、模型规模与泛化误差的三维等高线图。蓝点源自经验实验, 曲面是蓝点之间的线性插值。(图片来源: Rosenfeld et al. 2020)

因此, 他们可以构建一个简单参数函数形式的预测模型。利用 $\theta = \langle A, B, E, \alpha, \beta \rangle$ 来预测 N, D 大于特定阈值时的预期损失。训练过程只需在一组较小的训练配置 (N, D 小于特定阈值) 上进行即可。

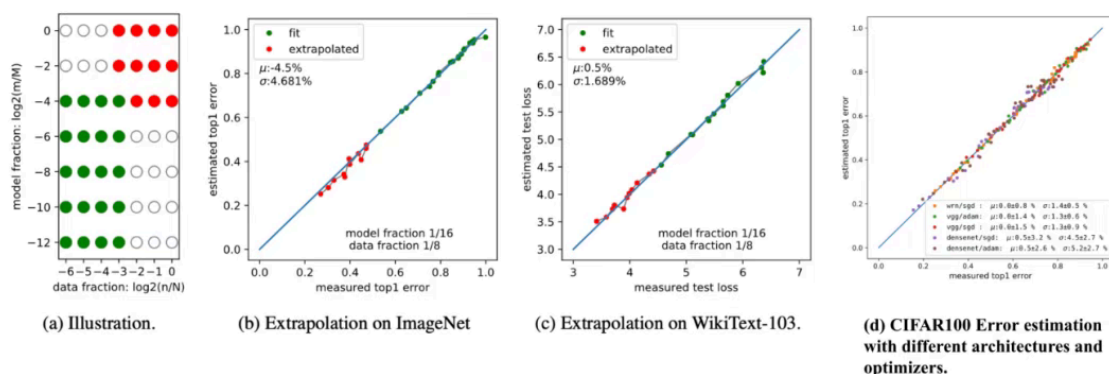


图4: 在小规模配置上拟合参数化误差模型, 并外推到更大规模的模型与数据体系: (a) 实验设置图解; (b) ImageNet、(c) WikiText-103 和 (d) CIFAR100 上的实验结果。包含三种架构 (WRN、VGG、DenseNet) 和两种优化器 (SGD、Adam) 的误差估计。(图片来源: Rosenfeld et al. 2020)

补充说明: 这些早期的工作依赖于经典的分类学习理论直觉, 例如将 VC 维度 (模型可以打散的最大点集的基数) 作为容量的代理。但在现代深度学习研究中, VC 维度往往过于粗糙, 难以解释模型的行为。事实证明, 经验性的幂律比理论提供的最坏情况边界清晰且实用得多。

数据无限区域的 Scaling Law

Kaplan 等人提出的 Scaling Law

Kaplan et al. (2020) 在语言建模社区普及了 Scaling law 的概念。他们发现交叉熵测试损失 L 分别与模型规模 N (不包括嵌入层)、数据集规模 D 和训练计算量 C 呈幂律缩放关系, 跨越了多

个数量级。这些发现与上一节中的早期工作保持一致。

Kaplan 等人将这一概念形式化，重点关注 Transformer 语言模型，并在更大规模上进行了实证实验。其实验模型规模从 7.68 亿到 15 亿非嵌入参数不等，数据集规模从 2200 万到 230 亿 token 不等。论文中的所有训练运行都使用了包含 3000 步线性预热的学习率调度，随后余弦衰减至零。

主要发现包括：

- 损失 L 与 N 、 D 和 C 分别呈幂律缩放。为了获得最佳性能，这三者必须同步扩展。
- 训练曲线遵循可预测的幂律，其参数大致独立于模型规模。
- 更大的模型具有更高的样本效率。这意味着，与小模型相比，大模型可以用更少的优化步数和数据点达到给定的损失水平。
- 架构细节（宽度、纵横比等）的重要性不及纯粹的规模。
- 训练损失和测试损失呈正相关。（这听起来显而易见，但却是预训练工作的基础。另一方面，预训练损失的改善是否能转移到后训练评估中，还需要单独的研究。）
- 在计算预算给定的情况下，训练一个超大模型并在收敛前停止，比将一个小模型训练到完全收敛更高效。这一发现与后续的 Chinchilla scaling law 存在分歧。由于拟合指数偏大，Kaplan 等人高估了最优模型规模。

他们将对 N 和 D 的联合依赖性总结在一个等式中：

$$\hat{L}(N, D) = \left[\left(\frac{a}{N} \right)^{\frac{\alpha}{\beta}} + \frac{b}{D} \right]^{\beta}$$

这种形式带来了一个很好的结论：过拟合的程度（即模型过于复杂或数据量过小）主要取决于比率 $N^{\alpha/\beta}/D$ 。这表明**数据需要按照与模型规模增长成特定比例的速度增长，以避免训练受到数据限制**。

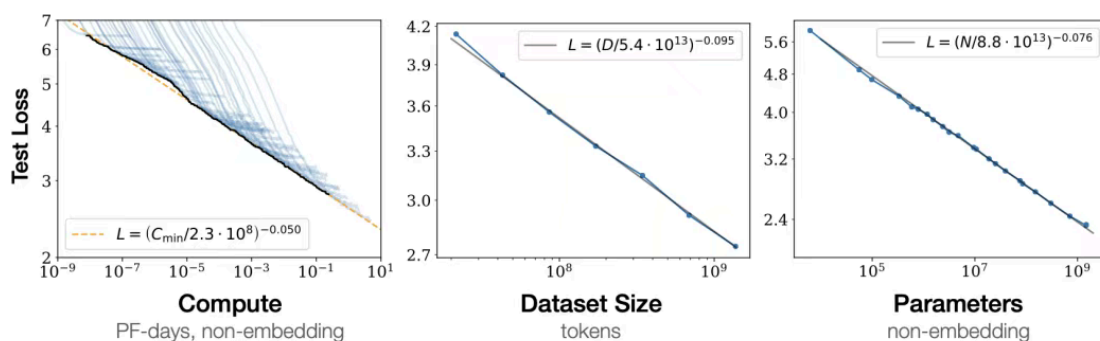


图5: 测试损失作为计算量、数据集规模和参数数量的幂律函数，跨越了多个数量级。（图片来源：Kaplan et al. 2020）

最具影响力、且在事后看来也备受争议的结论是计算最优分配。Kaplan 等人发现 $N_{\text{opt}} \propto C^{0.73}$ ，并且 $D_{\text{opt}} \propto C^{0.27}$ ，由此得出模型规模的增长速度应快于数据集规模。具体

来说，当计算量增加 10 倍时，他们建议将模型规模扩大约 5.5 倍，而训练 token 仅扩大约 1.8 倍。后来的 Chinchilla 论文推翻了这一建议，认为这种分配会导致大模型严重训练不足。

Kaplan 等人另一项有用的分析是根据 N 和 D 估算所需的训练 FLOPs。每次乘加运算计为大约 2 个 FLOPs。

Operation	Parameters	FLOPs per Token
Embed	$(n_{\text{vocab}} + n_{\text{ctx}}) d_{\text{model}}$	$4d_{\text{model}}$
Attention: QKV	$n_{\text{layer}} d_{\text{model}} 3d_{\text{attn}}$	$2n_{\text{layer}} d_{\text{model}} 3d_{\text{attn}}$
Attention: Mask	—	$2n_{\text{layer}} n_{\text{ctx}} d_{\text{attn}}$
Attention: Project	$n_{\text{layer}} d_{\text{attn}} d_{\text{model}}$	$2n_{\text{layer}} d_{\text{attn}} d_{\text{embd}}$
Feedforward	$n_{\text{layer}} 2d_{\text{model}} d_{\text{ff}}$	$2n_{\text{layer}} 2d_{\text{model}} d_{\text{ff}}$
De-embed	—	$2d_{\text{model}} n_{\text{vocab}}$
Total (Non-Embedding)	$N = 2d_{\text{model}} n_{\text{layer}} (2d_{\text{attn}} + d_{\text{ff}})$	$C_{\text{forward}} = 2N + 2n_{\text{layer}} n_{\text{ctx}} d_{\text{attn}}$

图6: 在给定层数 n_{layer} 、模型宽度 d_{model} (等于 d ；原始表格中的符号不一致)、前馈层维度 d_{ff} (通常相当于 $4d_{\text{model}}$)、注意力维度 d_{attn} (通常相当于 d_{model})、上下文长度 n_{ctx} 和词汇表大小 n_{vocab} 的情况下，对 Transformer 不同架构组件的参数数量和计算量进行估算。(图片来源: Kaplan et al. 2020)

给定一个标准配置，其中 $d_{\text{attn}} = d_{\text{model}} = d_{\text{ff}}/4$ ，并且在 N 和每 token 前向计算量中排除嵌入层：

$$\begin{aligned}
 N &= n_{\text{layer}} d_{\text{model}} 3d_{\text{attn}} + n_{\text{layer}} d_{\text{attn}} d_{\text{model}} + n_{\text{layer}} 2d_{\text{model}} d_{\text{ff}} && \text{; no embedding layer} \\
 &= 2 n_{\text{layer}} d_{\text{model}} (2d_{\text{attn}} + d_{\text{ff}}) \\
 &= 12 n_{\text{layer}} d_{\text{model}}^2
 \end{aligned}$$

$$\begin{aligned}
 C_{\text{fwd}} &= 2n_{\text{layer}} (d_{\text{model}} 3d_{\text{attn}} + n_{\text{ctx}} d_{\text{attn}} + d_{\text{attn}} d_{\text{embd}} + 2d_{\text{model}} d_{\text{ff}}) \\
 &= 2n_{\text{layer}} (12d_{\text{model}}^2 + n_{\text{ctx}} d_{\text{attn}}) \\
 &= 2N + 2n_{\text{layer}} n_{\text{ctx}} d_{\text{attn}} \\
 &\approx 2N \quad \text{; assuming } n_{\text{ctx}} < 12d_{\text{model}} \text{ and the } n_{\text{ctx}} \text{ term is relatively small.}
 \end{aligned}$$

然后，我们将反向传播的 FLOPs 计为前向传播 FLOPs 的两倍。因为反向传播需要运行两次矩阵乘法，分别用于计算输入激活和权重的梯度。因此，每个 token 的训练总 FLOPs 约为 $6N$ ，在 D 个 token 上训练的总 FLOPs 为 $C \approx 6ND$ 。

Chinchilla Scaling Law

Chinchilla 论文 (Hoffmann et al. 2022) 研究了在固定计算预算 C 下，最优模型规模 N (总参数量，包含嵌入层) 与 token 数量 D 之间的关系。该研究采用了更加严谨的实验设计，得出了与 Kaplan 等人有所不同的结论。



图7: 你应该知道龙猫 (chinchilla) 长什么样 😊 (图片来源: 由 ChatGPT 生成)

核心问题在于，在 $\text{FLOPs}(N, D) = C \approx 6ND$ 的约束下，分配资源的最佳策略是什么。换句话说，当我们只有有限的 FLOPs（例如特定数量的 GPU 运行特定时间）时，应该如何在增加数据 token 和增加模型参数之间做出选择？

$$N_{\text{opt}}(C), D_{\text{opt}}(C) = \underset{\text{s.t. } \text{FLOPs}(N, D)=C}{\arg \min} \hat{L}(N, D)$$

Chinchilla 论文提出了三种设计巧妙的 Scaling law 拟合方法。

实证实验扫描了 400 多个模型。这些模型的参数量从 7000 万到超过 160 亿不等，训练 token 数量从 50 亿到 5000 亿不等。这些实验基于一个假设：每个训练 token 都是独一无二的（即无限数据区域）。所有训练都采用了余弦学习率调度，在整个训练周期内衰减 10 倍。通过对不同模型规模的扫描，他们描绘出了计算最优边界。

方法 1: 固定模型规模，改变 token 预算

对于每个参数量 N ，使用不同的 token 预算进行多次训练。随后记录每个 FLOP 预算 C 下达到的最小损失。

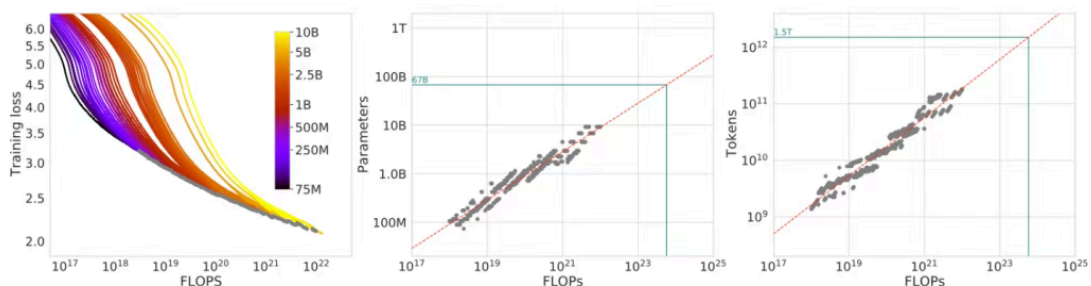


图8: Chinchilla 方法 1: 对一系列模型规模进行扫描时，不同 FLOP 预算下的训练损失曲线。(图片来源: Hoffmann et al. 2022)

方法 2: IsoFLOP 曲线

固定计算预算 C ，绘制最终损失与参数量 N 的关系图。在对数空间中，每条等 FLOP 曲线大致呈抛物线状。其最低点标志着该计算预算下的最优模型规模。然后，在不同的预算上重复这一过程，就能在图中描绘出一条幂律线。

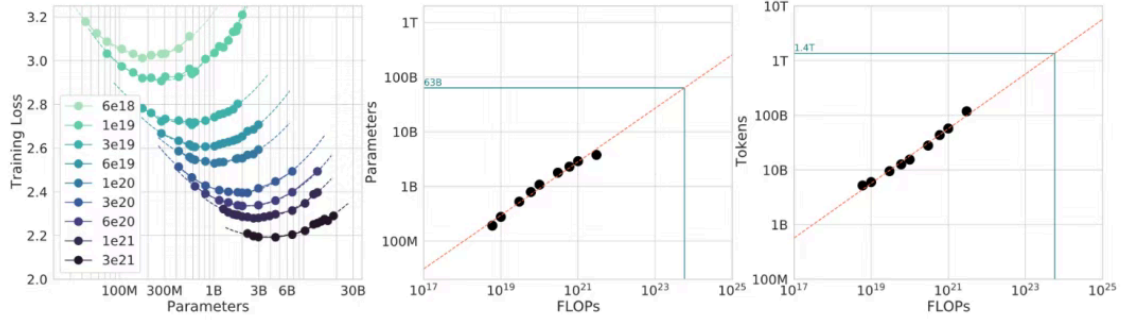


图9: Chinchilla 方法 2: IsoFLOP 抛物线；每条曲线的最低点代表了该预算下的计算最优模型规模。(图片来源: Hoffmann et al. 2022)

方法 3: 参数化拟合

直接拟合与 Rosenfeld et al. (2020) 相同的参数化函数：

$$\hat{L}(N, D) = \frac{A}{N^\alpha} + \frac{B}{D^\beta} + E$$

我们实际上可以通过在约束条件 $\text{FLOPs}(N, D) = C \approx 6ND$ 下最小化 $\hat{L}(N, D)$ ，获得最优 $N_{\text{opt}}(C), D_{\text{opt}}(C)$ 的闭式近似值。

首先，我们将表达式化简为仅包含 N 的形式：

$$\begin{aligned}\hat{L}(N) &= AN^{-\alpha} + B\left(\frac{C}{6}\right)^{-\beta} N^\beta + E \\ \hat{L}'(N) &= -\alpha AN^{-\alpha-1} + \beta B\left(\frac{C}{6}\right)^{-\beta} N^{\beta-1} = 0 \quad ; \text{derivative wrt } N \text{ should be zero.} \\ \text{Thus } \alpha AN^{-\alpha-1} &= \beta B\left(\frac{C}{6}\right)^{-\beta} N^{\beta-1} \\ \alpha A &= \beta B\left(\frac{C}{6}\right)^{-\beta} N^{\alpha+\beta} \\ N_{\text{opt}} &= \left(\frac{\alpha A}{\beta B}\right)^{\frac{1}{\alpha+\beta}} \left(\frac{C}{6}\right)^{\frac{\beta}{\alpha+\beta}} \\ D_{\text{opt}} &= \frac{C}{6N_{\text{opt}}} = \left(\frac{\beta B}{\alpha A}\right)^{\frac{1}{\alpha+\beta}} \left(\frac{C}{6}\right)^{\frac{\alpha}{\alpha+\beta}}\end{aligned}$$

当 $\alpha \approx \beta$ 时，模型规模和训练 token 应该以相同的速度扩展。

为了找到最优的 $\theta = \langle A, B, E, \alpha, \beta \rangle$ ，Chinchilla 论文采用了 Huber 损失（对异常值具有鲁棒性； $\delta = 10^{-3}$ ）和 L-BFGS 算法（适用于少量参数的曲线拟合）。

$$\min_{A,B,E,\alpha,\beta} \sum_{\text{runs } \{i\}} \text{Huber}_\delta(\log \hat{L}(N_i, D_i) - \log L_i)$$

$$\text{where } \text{Huber}_\delta(x) = \begin{cases} \frac{1}{2}x^2 & \text{for } |x| \leq \delta \\ \delta \cdot (|x| - \frac{1}{2}\delta), & \text{otherwise.} \end{cases}$$

Chinchilla 通过三种互补的方法得出了结论。这些方法的最终结果相互吻合，这也是该结果极具说服力的原因之一。

Approach	Coeff. a where $N_{\text{opt}} \propto C^a$	Coeff. b where $D_{\text{opt}} \propto C^b$
1. Minimum over training curves	0.50 (0.488, 0.502)	0.50 (0.501, 0.512)
2. IsoFLOP profiles	0.49 (0.462, 0.534)	0.51 (0.483, 0.529)
3. Parametric modelling of the loss	0.46 (0.454, 0.455)	0.54 (0.542, 0.543)
Kaplan et al. (2020)	0.73	0.27

图10: 这三种方法在计算最优边界上取得了一致，其中 $N_{\text{opt}} \propto C^{0.5}$ 但与 Kaplan 等人的结论存在分歧。值得注意的是，方法 3 的结果与其他两种方法略有偏差，我们稍后将对此进行解释。（图片来源：Hoffmann et al. 2022）

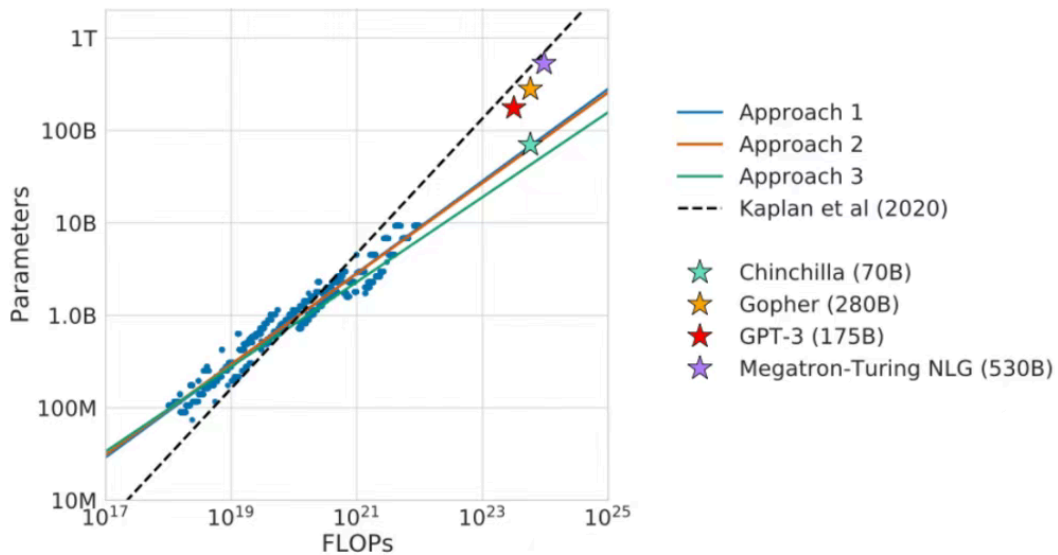


图11: Chinchilla 的三种不同方法的预测图，以及 Kaplan 等人 (2020) 的预测。这三种方法都表明，当时的一些主流大语言模型训练不足。（图片来源：Hoffmann et al. 2022）

Chinchilla 论文声称当时（约 2022 年）大多数大模型训练不足，这一主张得到了一次著名演示的支持。在与 Gopher（Rae et al. 2021；2800 亿参数，3000 亿 token 预算）相同的计算预算下，他们训练了 Chinchilla（700 亿参数，1.4 万亿 token 预算）。这是一个体积仅为四分之一，但训练 token 量大约是其四倍的模型。结果显示，Chinchilla 在各项指标上全面超越了 Gopher。

调和 Kaplan 与 Chinchilla 的分歧

Chinchilla scaling law 与 Kaplan 等人的研究在以下方面存在分歧：

- 相较于「模型增长快于数据增长」($N_{\text{opt}} \propto C^{0.73}$), 现在的做法更倾向于模型规模每翻一倍, 训练 token 的数量也应相应翻倍 ($N_{\text{opt}} \propto C^{0.5}$, $D_{\text{opt}} \propto C^{0.5}$)。
- 优先推荐在更多的数据上训练较小的模型。这一策略取代了先训练大模型然后在收敛前提前停止的做法。

两篇论文在原则上依然保持一致, 只是在最优规模与 token 的权衡点上存在分歧。为什么分歧会如此之大?

差异 1: Kaplan 等人主要在小模型上进行实验。 Kaplan 等人大多在较小的模型上进行测试, 而 Chinchilla 论文的实验规模要大 10 倍以上。当我们在双对数空间中进行外推时, 微小的拟合差异都可能导致结果出现巨大偏差 (参见模拟测试小节)。

差异 2: 嵌入层的参数量对小模型影响显著。 在小参数体系下, 嵌入参数在总参数量中占据了不可忽视的比例, 因此是否将其计算在内会产生影响。Pearce & Song (2024) 沿着这条思路进行了详尽的分析。我们用 $N_{\setminus E}, C_{\setminus E}$ 分别表示排除嵌入层后的模型规模和计算量, 用 N, C 表示总参数量。

- Kaplan 等人: $N_{\setminus E}^* \propto C_{\setminus E}^{0.73}$ (非嵌入层)
- Chinchilla: $N^* \propto C^{0.50}$ (总参数)

为了衔接这两种方法, 他们为总参数量 N_T 和非嵌入层参数量 $N_{\setminus E}$ 拟合了一个关系:

$N = N_{\setminus E} + \omega N_{\setminus E}^{1/3}$, 其中 ω 为常数。这种形式具有严格递增的良好性质。并且对于大型模型来说, $\lim_{N \rightarrow \infty} N = N_{\setminus E}$ (因为 $\frac{N}{N_{\setminus E}} = 1 + \omega N_{\setminus E}^{-2/3}, \lim_{N_{\setminus E} \rightarrow \infty} \frac{N}{N_{\setminus E}} = 1$)。

将此代入 Chinchilla 定律的等式中:

$$\begin{aligned}
 L(N_{\setminus E}, C_{\setminus E}) &= A(N_{\setminus E} + \omega N_{\setminus E}^{1/3})^{-\alpha} + B\left(\frac{C_{\setminus E}}{6}\right)^{-\beta} N_{\setminus E}^{\beta} + E \\
 L'(N_{\setminus E}, C_{\setminus E}) &= -\alpha A(N_{\setminus E} + \omega N_{\setminus E}^{1/3})^{-\alpha-1} \left(1 + \frac{\omega}{3} N_{\setminus E}^{-2/3}\right) + \beta B\left(\frac{C_{\setminus E}}{6}\right)^{-\beta} N_{\setminus E}^{\beta-1} = 0 \quad ; \text{ derivative wrt } N_{\setminus E} \text{ should be zero.} \\
 \text{Rearrange to get } &\alpha A(N_{\setminus E}^* + \omega N_{\setminus E}^{*1/3})^{-\alpha-1} \left(1 + \frac{\omega}{3} N_{\setminus E}^{*-2/3}\right) = \beta B\left(\frac{C_{\setminus E}}{6}\right)^{-\beta} N_{\setminus E}^{*\beta-1} \\
 6^{-\beta} \frac{\alpha A}{\beta B} (N_{\setminus E}^* + \omega N_{\setminus E}^{*1/3})^{-\alpha-1} \left(1 + \frac{\omega}{3} N_{\setminus E}^{*-2/3}\right) N_{\setminus E}^{*1-\beta} &= C_{\setminus E}^{-\beta} \\
 6 \left(\frac{\beta B}{\alpha A}\right)^{\frac{1}{\beta}} (N_{\setminus E}^* + \omega N_{\setminus E}^{*1/3})^{\frac{1+\alpha}{\beta}} (N_{\setminus E}^* + \frac{\omega}{3} N_{\setminus E}^{*1/3})^{-\frac{1}{\beta}} N_{\setminus E}^* &= C_{\setminus E}
 \end{aligned}$$

上述等式中 $C_{\setminus E}$ 与 $N_{\setminus E}$ 之间的关系不再是单纯的幂律。我们只能在局部将其近似为 $N_{\setminus E}^* \propto C_{\setminus E}^g$ 。这里的 g 是基于 $\sim$ 计算出的局部指数, 而非全局幂律指数。由此会得出 $g = \frac{d \log C_{\setminus E}}{d \log N_{\setminus E}}$ 。有关指数近似方式的完整细节, 请参阅 Pearce & Song (2024) 的附录 A.1。

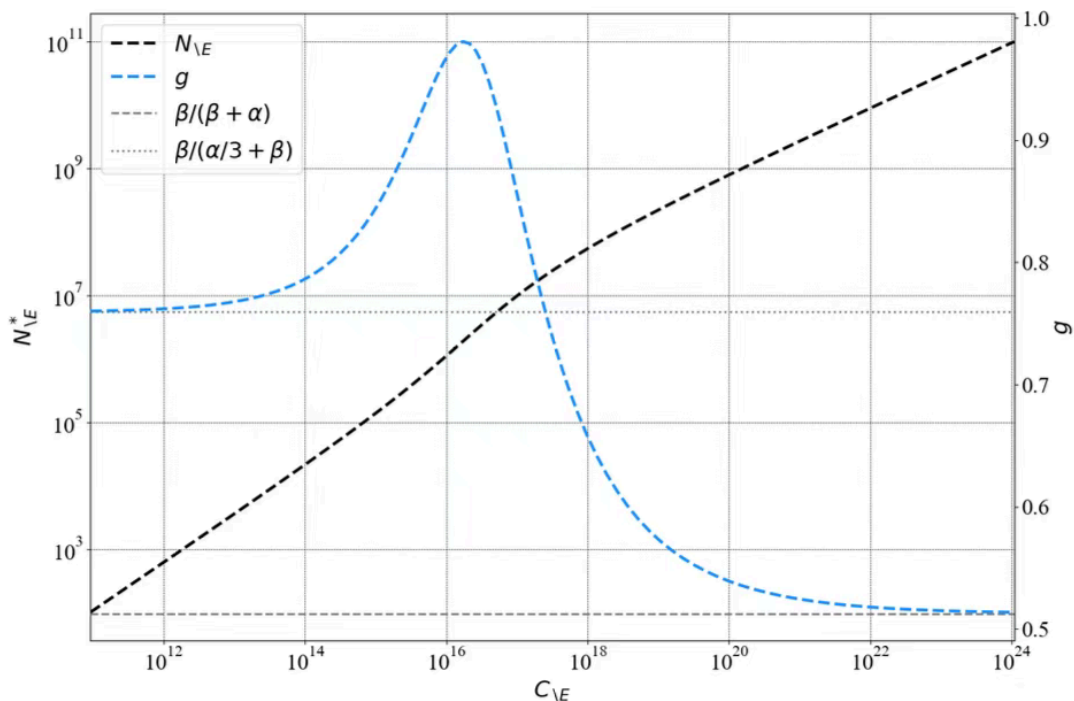


图12: 局部幂律指数随 C_E 增长的可视化。(图片来源: Pearce & Song 2024)

如上图所示，随着 C_E 变大， g 会收敛到 Chinchilla 的估计值。通过利用上述等式生成合成的训练曲线，在 7.68 亿到 15 亿的模型规模范围内（正如 Kaplan 等人所做），他们估算出 g 接近 Kaplan 在该区域给出的 0.73 系数。

为什么是幂律？

幂律现象在人工智能以外的许多领域都能被广泛观察到。例如齐夫定律（Zipf's law）、无标度网络、城市缩放定律以及许多其他复杂系统。反复出现的模式是：大事件少见，小事件常见，且规模与频率之间的关系在双对数尺度上通常呈现为一条直线。

为什么大语言模型的 Scaling law 也呈现出幂律的形式？

这一现象部分是受到了不同领域展现出不同指数（Hestness et al. 2017）的启发。Sharma & Kaplan (2020) 提出了一种早期解释。他们假设语言建模可以被视为在一个低维数据流形上进行回归。更多的模型参数能够促使数据流形产生更细致的划分，从而带来更小的泛化误差。简单来说，如果一个有效规模为 N 的模型将 d 维流形划分为 $O(N)$ 个区域，那么典型的线性分辨率会按 $\sim N^{-1/d}$ 的比例缩放。这与上述 Scaling law 具有相似的幂律形式。这一理论最完美地适用于数据无限、欠拟合的状态。但在现实中，估算数据流形的固有维度是一件非常困难的事。

后续的一个假说（Michaud et al. 2023, Brill 2024）认为，知识或技能是以离散块的形式进行学习的（即「量化」），且这些技能的频率分布遵循幂律。模型会先学习常见的技能，随后再学习罕见的技能。这一过程使得损失呈现平滑的幂律衰减。

我在这里仅列举了两种假设。此外还有更多研究尝试通过数据的谱尾、核特征值、自然语言统计特征，或训练动态中的相变现象来解释幂律缩放的形态。

数据受限区域的 Scaling Law

经典的 Scaling law 假设存在几乎无限的去重数据，没有重复，也没有多轮次训练。随着模型规模的显著增长，我们逐渐面临高质量去重 token 数量不足的问题。事实上，关于人工智能的规模化还能持续多久的争论，核心焦点往往在于我们是否触及了「数据墙」。

另外值得强调的是，D 背后所代表的数据集应该是已经清理过的。预训练数据管道通常是高效预训练流程的重要组成部分。常见的步骤包括去重（精确去重和模糊去重）、质量过滤、样板去除、安全过滤、个人信息 / 版权脱敏、基准污染处理，以及根据语言、质量、内容类型等对数据混合成分进行仔细重加权。即使两个数据集包含相同数量的 token 数 D，一个高质量的数据集和一堆互联网垃圾数据产生的计算效率也可能截然不同。

Hernandez et al. (2022) 的研究专注于一种受控场景：一个包含小比例重复数据的数据集。他们从一个大型数据集开始配置数据混合比例，保留 90% 的去重数据，并将剩余的 10% 替换为原始数据集一小部分的重复内容。在用 1000 亿个 token 训练 Transformer 模型后，他们观察到了双重下降现象。测试损失会随着重复数据占比的增加而变差，然后再变好。重复数据的比例越高，这种影响就越明显。

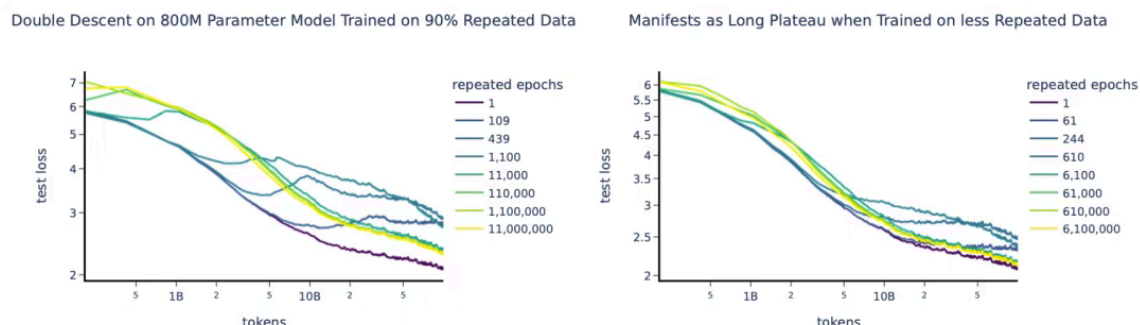


图 13. 随着重复率增加，测试损失中出现双重下降现象（左图重复率为 90%，右图重复率为 50%）。（图片来源：Hernandez et al. 2022）

训练中期出现的平滑或上升趋势，可能是由于模型记住了重复的数据。这种形态的学习曲线会降低 Scaling law 拟合的准确性。研究人员还得出结论，重复数据会损害某些分布外（OOD）评估和下游微调的效果。然而，他们的数据组合方式建立在偏向实验室环境的设定上。在真实世界的的数据中，重复情况通常更加复杂微妙（例如，不同数据存在着不同程度的重复、语义上的重复等）。

考虑到高质量的去重数据并非无穷无尽，我们在训练期间不可避免地需要重复使用数据。相较于探讨数据重复对训练的不利影响，我们更关注在这样的条件下应该如何拟合 Scaling law。

Muennighoff et al. (2023) 探讨了当模型训练受到数据限制时，应该如何最优分配计算资源。具体而言，他们通过大约 400 次实验实证研究了数据重复带来的影响。实验涵盖了 1000 万到 90 亿参数，数据量高达 9000 亿个 token，训练轮次高达 1500 轮。在每个轮次中都重复完全相同的数据集，在轮次之间进行洗牌，并在独立的测试集上进行评估。

关键的建模调整在于将总 token 数 D 分解为两部分：(i) 去重 token 的数量 U_D

和 (ii) 重复的次数 R_D （即训练轮次减 1）。因此我们可以得出 $D = U_D(1 + R_D)$ 。在给定的去重数据预算 D_{uniq} ，根据定义 $U_D = \min\{D_{\text{uniq}}, D\}$ 和 $R_D = (D/U_D) - 1$ 。他们使用 Chinchilla scaling law 来找到拟合 U_D 的最佳模型大小 U_N ，并通过重复次数 $R_N = (N/U_N) - 1$ 定义多余的模型规模。

随后，他们对 Chinchilla 的参数化拟合（方法 3）进行了更新，采用有效（折减后）数据量 D' 与模型规模 N' 替代原始数值：

$$\hat{L}(N, D) = \frac{A}{N'^{\alpha}} + \frac{B}{D'^{\beta}} + E$$

其中，

$$D' = U_D + U_D r_D \left(1 - \exp\left(-\frac{R_D}{r_D}\right)\right)$$

其背后的直觉是，token 的价值在被重复使用时呈指数衰减。在他们的模型中，每次重复都会使代币损失其剩余价值的 $(1 - 1/r_D)$ 倍，其中 r_D 是一个可学习的「半衰期」参数。当 $R_D = 0$ or $R_D \ll r_D$ 时，我们就能恢复出 $D' \approx D$ 。

对称公式处理了模型过大的问题：

$$N' = U_N + U_N r_N (1 - \exp(-R_N/r_N))$$

它体现了「模型越大，重复数据上的过拟合速度越快」以及「模型可能对于其数据集而言过大」的观点。这一部分不太直观，我也没有找到令人满意的解释来说明为什么模型大小需要以这种对称形式出现在重复数据中。Lovelace 等人(2026)的后续研究改变了这一假设。

他们的实证拟合结果表明，多余参数在价值上的衰减速度快于重复数据 $r_N < r_D$ 。这表明我们在分配资源时，应该优先考虑增加训练轮次，而不是一味扩大模型规模。这种建模方式的弱点正如作者所指出的，它严重低估了出现失败的模型的最终测试损失（即训练中途损失开始增加的模型），例如那些训练了 44 个轮次的模型。

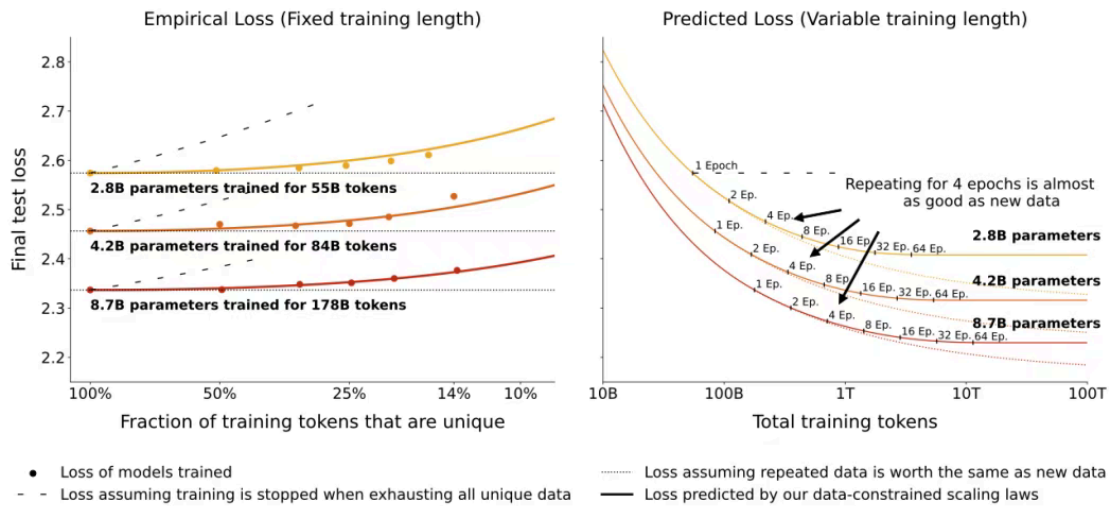


图14: 在数据受限并带有重复数据的情况下进行缩放, 比忽略数据重复情况的拟合能更好地捕捉实验结果。重复 token 的价值会向着一个上限呈现指数衰减。随着轮次增加, 由于高度重复导致测试损失在训练中途增加 (图中未显示), 拟合效果会变差。(图片来源: Muennighoff et al. 2023)

最近, Lovelace et al. (2026) 用一种不同的方法重新审视了这个问题。与其将过度参数化建模为对有效模型规模的递减收益, Lovelace 等人选择对模型规模与数据重复之间的相互作用进行显式建模。在实证研究中, 他们训练了大约 300 个模型, 参数范围从 1500 万到 10 亿, 去重 token 数量从 5000 万到 60 亿。

当他们在不同的数据重复水平下绘制固定规模模型的拟合残差图时, 观察结果非常直观: 轮次越多, 造成的损害就越大。有趣的是, 模型规模越大, 对重复数据就越敏感。这暗示损失惩罚可能是模型规模和数据规模的共同函数。

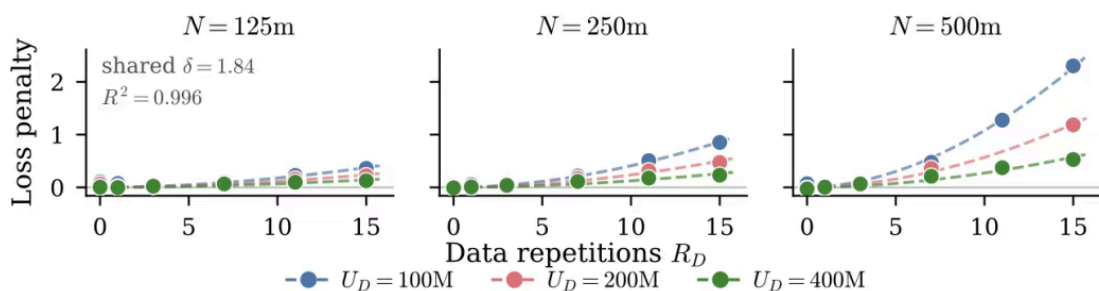


图15: 有效规模拟合的残差显示, 过拟合造成的损害会随着轮数和模型规模的增加而增大。(图片来源: Lovelace et al. 2026)

研究人员引入了一个显式的过拟合惩罚项, 它建立在容量比 (参数量相对于去重 token 数量的比例, N/D_U) 的基础之上:

$$\hat{L}(N, U_D, R_D) = E + \frac{A}{N^\alpha} + \frac{B}{(U_D(1 + R_D))^\beta} + P \cdot R_D^\delta \cdot \left(\frac{N}{U_D}\right)^\kappa$$

其中:

- R_D 是重复次数；
- 标量 P 是可学习参数；
- 指数 κ （第二个可学习参数）使得惩罚项能随容量比 N/U_D 非线性地缩放；
- 单独作用在重复次数上的指数 δ （第三个可学习参数）解耦了重复带来的非线性惩罚与 κ 的关系。

增加的（红色部分）一项是直接的过拟合惩罚。这种惩罚会随着数据重复次数的增加，以及相对于可用去重数据模型过度参数化程度的增加而变得更加严重。

他们还进行了一项案例研究，探讨了在数据受限的条件下权重衰减对训练的影响。研究发现，强烈的权重衰减能减少因数据重复而引起的过拟合惩罚。

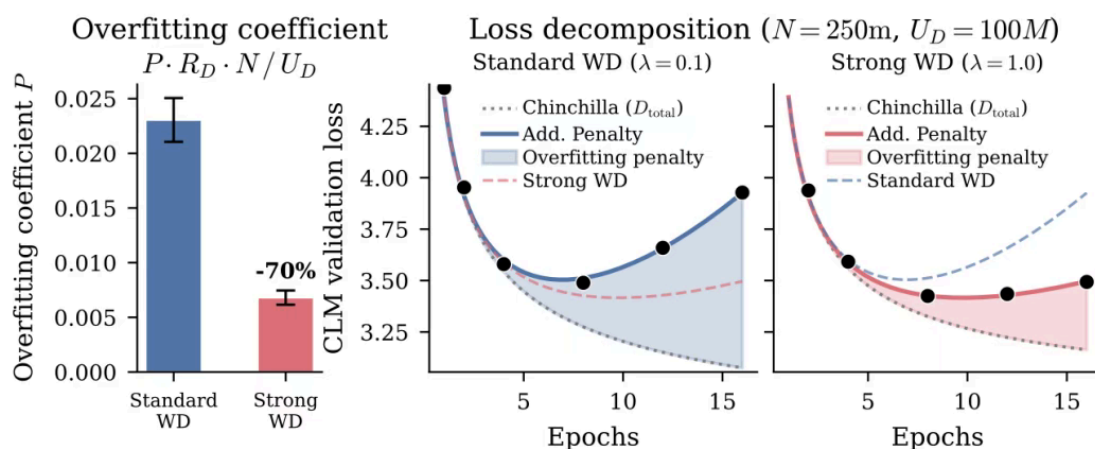


图16: 强烈的权重衰减能减轻数据重复带来的过拟合惩罚。(图片来源: Lovelace et al. 2026)

Muennighoff 等人和 Lovelace 等人提出的两种建模方法都是建立在经验曲线拟合的基础上的。因此目前仍不清楚为什么受数据限制的 Scaling laws 必须采用这些特定的形式，以及为什么需要引入这些自由参数。期待这一领域能有更多相关的理论研究。

在现实中拟合 Scaling Law 的微妙之处

尽管形式上很简洁，但在实践中，Scaling law 的拟合却对一些看似微不足道的流程选择出人意料地敏感。这包括如何计算参数量、如何进行精度四舍五入，以及如何求和或平均损失等操作。

这是因为 Scaling law 通常是在我们负担得起的（相对较小、成本较低的）模型上拟合出来的，而相关预测则是要外推到规模大上几个数量级的模型。在这样的设定下，看似舍入误差的细微选择，都可能导致预测结果大相径庭。

同时，Scaling law 的拟合假设唯一的变量就是规模。这意味着模型架构、优化器、学习率调度、批量大小预热、数据混合、分词器和其他设计选择都应保持不变。另一个潜在的假设是，所有这些设置都已经经过了仔细的调整。因为像模型训练不足这样的情况会导致截然不同的结论。

Kaplan 等人与 Chinchilla 的结果存在分歧，就是一个证明 Scaling law 拟合具有微妙之处的例子。

第二个例子是一项后续的分析。该分析调查了为什么 Chinchilla 的方法 3 与其他两种方法存在轻微偏差。Besiroglu et al. (2024) 从 Hoffmann et al. (2022) 论文的图 4 中提取了原始 (N, D, L) 数据点，并重新运行了方法 3 的参数化拟合。他们发现了几个具体的问题：

- L-BFGS-B 最小化器中的损失刻度过高。这主要是因为对样本的 Huber 损失值进行了求平均而非求和。这导致了优化过程过早终止。在原始拟合和 bootstrapping 重采样过程中，损失最小化提前停止，这产生了不一致的估计值和令人难以置信的狭窄置信区间。
- 报告的 α 和 β 被四舍五入到了 2 位精度。这使得推导出的 A,B 看起来比实际偏差更大。

模拟测试

这是一个由 ChatGPT 创建的模拟测试小部件，旨在演示三种特定的故障模式。

我们假设真实函数为：

$$\hat{L}(N, D) = 482.01 \cdot N^{-0.3478} + 2085.43D^{-0.3658} + 1.8172$$

因此，

$$N_{\text{opt}} \propto C^{0.5126}, D_{\text{opt}} \propto C^{0.4874}$$

这是 Besiroglu et al. (2024). 的估计。

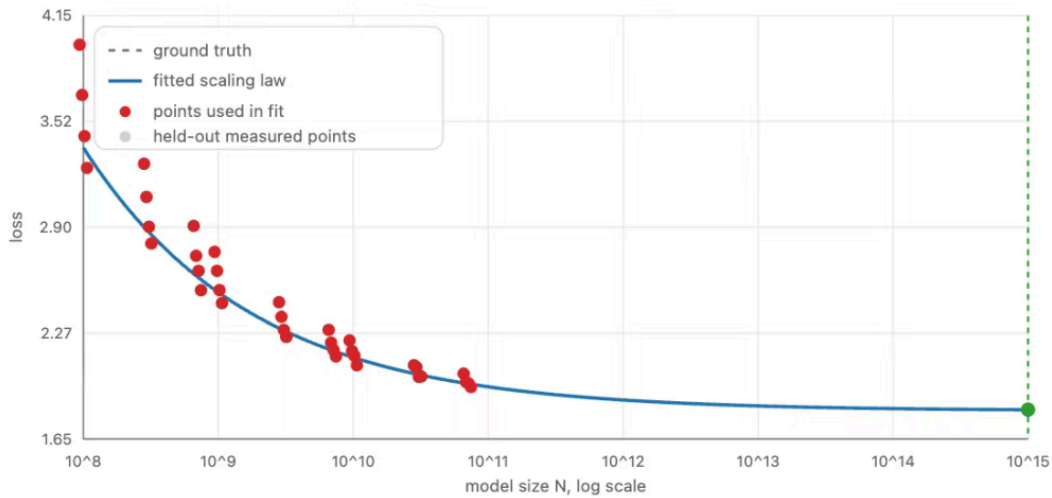
该模拟绘制了损失预测值 $\hat{L}$ 与数据集大小 D 的关系图，同时提供了一组 slider 来展示案例：

- 损失精度：将损失值从高位小数四舍五入到低位小数可能会改变拟合参数值。
- 损失噪声：仅以千分之一损失（0.001）为倍率扰动损失值，就会产生不同的拟合效果。
- 拟合区域敏感性：仅拟合小型模型、仅拟合中型模型或拟合全部模型，会得出不同的 scaling law。

Scaling-law fitting playground

Ground truth: $\mathcal{L}(N,D) = 1.8172 + 482.01N^{-0.3478} + 2085.43D^{-0.3658} + 1.8172$.

Thus $N_{\text{opt}} \propto C^{0.5126}$ and $D_{\text{opt}} \propto C^{0.4874}$.



Fit region

All measured models

Changing the fitted scale range changes the apparent exponents.

Loss precision

4 decimals



Round observed losses before fitting.

Loss noise

-5 mL



Applies a correlated loss-estimation bias across the fitted region.

Extrapolate to

N=10^15



Prediction is evaluated at $D = 20N$.

Red points are used in the fit; gray points are held out. Points at the same model size use different token budgets. The dashed gray curve is the synthetic ground truth..

Fit result

Fitted α	0.3461
Fitted β	0.3650
Fitted E	1.8148
Fitted N_{opt} exp.	0.5133 / true 0.5126
Fitted D_{opt} exp.	0.4867 / true 0.4874
Predicted loss	1.8206
True loss	1.8229
Error	-0.0023

原图为交互图，请移步原文体验交互效果。

参考文献：

- [1] S. Amari, N. Fujita, and S. Shinomoto. "Four Types of Learning Curves. *Neural Computation*." 4 (4):605–618, 1992.
- [2] Hestness et al. "Deep Learning Scaling is Predictable, Empirically." *arXiv preprint arXiv:1712.00409*, 2017.
- [3] Rosenfeld et al. "A Constructive Prediction of the Generalization Error Across Scales." *ICLR* 2020.
- [4] Kaplan et al. "Scaling Laws for Neural Language Models." *arXiv preprint arXiv:2001.08361*, 2020.
- [5] Hoffmann et al. "Training Compute-Optimal Large Language Models." *NeurIPS* 2022.

- [6] Pearce and Song. "Reconciling Kaplan and Chinchilla Scaling Laws." TMLR 2024.
- [7] Bahri et al. "Explaining Neural Scaling Laws." arXiv preprint arXiv:2102.06701, 2021.
- [8] Sharma and Kaplan. "A Neural Scaling Law from the Dimension of the Data Manifold." arXiv preprint arXiv:2004.10802, 2020.
- [9] Hernandez et al. "Scaling Laws and Interpretability of Learning from Repeated Data." arXiv preprint arXiv:2205.10487, 2022.
- [10] Muennighoff et al. "Scaling Data-Constrained Language Models." NeurIPS 2023.
- [11] Lovelace et al. "Prescriptive Scaling Laws for Data Constrained Training." arXiv preprint arXiv:2605.01640, 2026.
- [12] Besiroglu et al. "Chinchilla Scaling: A Replication Attempt." arXiv preprint arXiv:2404.10102, 2024.
- [13] Michaud et al. "The Quantization Model of Neural Scaling" NeurIPS 2023.
- [14] Brill. "Neural Scaling Laws Rooted in the Data Distribution." arXiv preprint arXiv:2412.07942, 2024.
- [15] Rae et al. "Scaling Language Models: Methods, Analysis & Insights from Training Gopher." arXiv preprint arXiv:2112.11446, 2021.